

神经网络的模型与应用作业论文阅读笔记

Unified Stabilization Approach to Principal and Minor Components Extraction Algorithms

1 论文基本信息与主要内容

本文作者为 Tianping Chen 和 Shun-ichi Amari, 发表在 *Neural Networks*. 文章讨论 principal component analysis, 即 PCA, 和 minor component analysis, 即 MCA, 的连续时间学习算法稳定性问题.

PCA 的目标是从协方差矩阵 $C = \mathbb{E}[xx^T]$ 中提取最大特征值对应的特征向量或主子空间. MCA 则相反, 它要提取最小特征值对应的特征向量或子空间. 这两类方法在神经网络学习, 信号处理, 降维和噪声消除中都很重要.

文章关心的问题不是 PCA/MCA 的线性代数定义, 而是如下更细的问题:

为什么某些 PCA 动力系统算法不能简单改变符号就得到稳定的 MCA 算法?

文章的回答是: 关键在于相关约束流形的稳定性. 如果算法只在约束流形内部稳定, 但流形本身在法向扰动下不稳定, 那么离散迭代或数值扰动会使算法发散. 作者通过加入稳定化项, 给出统一的 PCA/MCA 稳定化框架.

2 PCA, MCA 与子空间提取

2.1 PCA 与 MCA 的基本目标

设协方差矩阵 C 的特征值分解为

$$C = P\Lambda P^T, \quad \Lambda = \text{diag}(\lambda_1, \lambda_2, \dots, \lambda_n), \quad (1)$$

其中可按

$$0 < \lambda_1 < \lambda_2 < \dots < \lambda_n \quad (2)$$

排列. PCA 关心最大特征值对应的方向, MCA 关心最小特征值对应的方向.

若只求 p 维主子空间, 那么只要得到由前 p 个主方向张成的空间即可, 不一定要区分每一个具体特征向量. 但若要求 individual principal components, 就必须把每个特征向量也分开. 文章指出, 后者比前者更难, 因为需要处理更多的稳定性和正交约束问题.

2.2 Oja-Karhunen 主子空间算法

早期主子空间提取算法可以写成

$$\dot{M} = CM - MM^T CM, \quad (3)$$

其中 $M \in \mathbb{R}^{n \times p}$. 该算法能让 M 的列向量逐渐张成主子空间, 但它通常不直接保留各个特征向量和特征值的个体信息.

2.3 Xu 算法与 Stiefel 流形

为了提取具体的 p 个特征向量, Xu 算法引入对角矩阵

$$D_p = \text{diag}(d_1, \dots, d_p), \quad d_1 > d_2 > \dots > d_p > 0, \quad (4)$$

并考虑

$$\dot{Q} = CQD_p^2 - QD_p^2Q^T CQ, \quad Q^T Q = I_p. \quad (5)$$

这里 Q 位于 Stiefel 流形上, 即列向量两两正交且长度为 1. 对角矩阵 D_p 的不同权重打破了子空间内部的旋转不唯一性, 因而可以选出具体的特征向量.

注 2.1. 这也解释了为什么“求子空间”比“求具体特征向量”容易. 子空间内部允许任意正交变换, 而具体特征向量提取必须消除这种不唯一性.

3 符号反转为什么会失败

一种自然想法是: PCA 是最大化某个目标函数, MCA 是最小化同一个目标函数, 那么把 PCA 算法右端换一个负号是否就能得到 MCA 算法? 文章指出, 这个想法一般是错误的.

例如把 Oja 型算法简单反号, 会得到

$$\dot{M} = -CM + MM^T CM. \quad (6)$$

把 Xu 算法反号, 会得到

$$\dot{Q} = -CQD_p^2 + QD_p^2Q^T CQ. \quad (7)$$

这些算法在形式上像是 MCA, 但数值模拟会出现发散. 原因不是目标函数方向错了这么简单, 而是约束流形本身的法向稳定性变坏了. 也就是说, 一旦由于离散化误差或随机扰动离开 $Q^T Q = I_p$ 或 $M^T M = D_p^2$ 这类流形, 系统不会自动回到流形, 反而可能进一步偏离.

4 自然梯度算法与不变流形

4.1 一对自然梯度算法

文章重点讨论了如下形式的一对算法:

$$\dot{M} = CMM^T M - MM^T CM, \quad (8)$$

$$\dot{M} = -(CMM^T M - MM^T CM). \quad (9)$$

第一个用于提取 principal components, 第二个用于提取 minor components. 这对算法的形式非常对称, 看起来正好可以通过改变符号实现 PCA 与 MCA 的对偶.

若记 $M(t)$ 的奇异值分解为

$$M(t) = Q(t)D(t)R^T(t), \quad (10)$$

则文章指出, 对连续时间系统而言, $M^T(t)M(t)$ 会保持在初始给定的流形上. 记

$$\mathcal{S} = \{M : M^T M = E\}, \quad E = M^T(0)M(0). \quad (11)$$

在这个流形内部, 算法具有良好性质. 但是这只是中性稳定, 不是严格稳定. 中性稳定意味着小扰动不会立刻被放大, 但也不会被主动拉回. 对离散算法来说, 这种中性稳定往往不够安全.

4.2 稳定化思想

为了解决这个问题, 作者加入惩罚项, 使系统偏离流形时被拉回. 典型形式为

$$\dot{M} = CMM^T M - MM^T CM + M(E - M^T M), \quad (12)$$

以及

$$\dot{M} = -(CMM^T M - MM^T CM) + M(E - M^T M). \quad (13)$$

其中 $M(E - M^T M)$ 就是稳定化项. 当 $M^T M = E$ 时, 这一项为 0, 算法在流形内部与原算法一致; 当 $M^T M$ 偏离 E 时, 该项把系统拉回到约束流形附近.

注 4.1. 这类似于在优化算法中加入一个“软约束恢复项”. 它不改变流形上的目标运动, 但改善了流形外的稳定性.

4.3 在流形内部的约化形式

若 $E = D_p^2$, 并且 $M^T M = D_p^2$, 则上述稳定化 PCA 算法可以约化为

$$\dot{M} = CMD_p^2 - MM^T CM. \quad (14)$$

对应的 MCA 算法为

$$\dot{M} = -(CMD_p^2 - MM^T CM). \quad (15)$$

这说明稳定化项主要负责法向稳定, 而真正的 PCA/MCA 提取方向仍由主项决定.

5 稳定性分析的主要思路

5.1 对角化与坐标变换

为了分析稳定性, 文章将协方差矩阵对角化:

$$C = P\Lambda P^T. \quad (16)$$

再令

$$W = P^T M V, \quad (17)$$

其中 V 来自初始矩阵的奇异值分解. 经过这个正交变换后, 可以把问题化到 $C = \Lambda$ 且约束为

$$W^T W = D_p^2 \quad (18)$$

的标准形式.

5.2 切向扰动与法向扰动

在流形

$$\mathcal{S} = \{W : W^T W = D_p^2\} \quad (19)$$

上, 扰动可以分成两类:

- (1) 切向扰动: 仍然沿着约束流形运动, 主要改变子空间或基的选择.
- (2) 法向扰动: 使 $W^T W$ 偏离 D_p^2 , 也就是破坏正交或长度约束.

文章证明, 对稳定化算法, 若法向扰动写成 $\delta W = WG$, 其中 G 是对称矩阵, 则大致满足

$$\frac{dG_{ij}}{dt} = -2(d_i^2 + d_j^2)G_{ij}. \quad (20)$$

因此 G_{ij} 会指数趋于 0, 这说明流形 $\mathcal{S}$ 在法向方向是严格稳定的.

5.3 平衡点稳定性

在标准坐标中, 正确的 PCA 平衡点对应于 W 的列向量落在最大特征值对应的特征方向上; 正确的 MCA 平衡点对应于最小特征值对应的方向上. 文章通过线性化矩阵的迹和行列式判断稳定性.

对于一个 2×2 的扰动块 A_{ij} , 文章证明在正确平衡点附近有

$$\text{tr}(A_{ij}) < 0, \quad \det(A_{ij}) > 0. \quad (21)$$

这说明该二维扰动块的两个特征值实部都为负. 而对于错误的平衡点, 可以找到某个方向使扰动增长, 所以错误平衡点不稳定.

6 算法行为的统一解释

文章最后给出了一张表, 总结不同算法的动力学行为. 其核心结论可以概括为:

- (1) Oja 型和 Xu 型算法对 PCA 是稳定的, 但简单反号后用于 MCA 时, 约束流形可能变成不稳定.
- (2) 自然梯度算法的原始连续时间形式在 $M^T M = D_p^2$ 流形上是中性稳定的, 因而理论上可行, 但数值离散时容易出现边际不稳定.
- (3) 加入 $M(E - M^T M)$ 这样的稳定化项后, PCA 和 MCA 可以分别得到稳定算法.
- (4) 稳定化项的作用不是改变主成分或次成分的目标, 而是确保偏离约束流形时能够自动返回.

从这个角度看, 本文题目中的 “unified stabilization approach” 指的是: 不再逐个为 PCA 和 MCA 设计互不相关的算法, 而是在同一类自然梯度算法中加入统一的流形稳定化机制.

7 数值模拟的意义

文章使用三维高斯随机向量做数值模拟, 其协方差矩阵有给定的特征分解. 模拟比较了稳定化 MCA 算法, 原始 MCA 算法, 以及 Oja 型算法的表现.

结果表明:

- (1) 稳定化算法可以使权重向量收敛到目标特征向量方向.
- (2) 未稳定化的符号反转算法可能出现权重范数持续增大, 最终发散.
- (3) 对于主成分提取, 稳定化算法可以提取 individual principal components, 而普通 Oja 子空间算法更多只给出主子空间.

这些模拟与前面的稳定性分析是一致的. 它们说明本文讨论的稳定性不是纯粹形式问题, 而是会直接影响学习算法在离散实现中的可靠性.

8 个人理解与评价

8.1 本文的主要贡献

我认为本文最重要的贡献是把 PCA/MCA 算法的成功或失败解释为动力系统的流形稳定性问题. 很多算法在理想流形上看起来正确, 但真实计算中只要稍微偏离流形, 稳定性差异就会显现出来.

这篇文章还说明, PCA 和 MCA 的关系不能简单理解为目标函数正负号相反. 对于带约束的矩阵动力系统, 目标函数方向, 切向稳定性和法向稳定性必须同时考虑.

8.2 与神经网络学习的联系

从神经网络角度看, M 可以理解为权重矩阵, PCA/MCA 提取对应无监督学习中的特征学习. 本文的稳定化项类似于让网络权重自动保持归一化和正交性. 如果没有这种机制, 权重可能因为数值误差逐渐偏离可解释的特征方向.

8.3 本文的局限

本文主要分析连续时间微分方程模型, 虽然也讨论了离散迭代中的问题, 但对有限步长随机逼近的严格收敛分析不是全文重点. 实际机器学习中, 数据是有限样本, 步长不是无穷小, 噪声也可能非高斯, 因此还需要进一步考虑随机近似误差和步长选择问题.

8.4 我的整体理解

本文给我的启发是, 一个学习算法不仅要在目标函数意义下方向正确, 还要在几何约束意义下稳定. 对 PCA/MCA 这类问题而言, 正交约束和尺度约束不是附属细节, 而是算法能否稳定工作的核心. 稳定化项的思想也可以推广到其他带约束的神经网络学习算法中.